Desenvolvimento Web com .NET

Aula 10 – JWT

Prof. Me. Lucas Teodoro dos Santos

UNIP

28 de outubro de 2026

Roteiro da Aula

1. Introdução ao JWT
2. Anatomia de um JWT
3. Implementação com JWT

UNIP
UNIVERSIDADE PAULISTA

O que é JWT?

Um crachá digital para acessar uma aplicação

JWT significa *JSON Web Token*. Ele é usado por muitos sistemas para representar o acesso de um usuário após o login.

- Você entra no sistema com seu usuário e sua senha.
- O sistema entrega um token, que funciona como um **crachá digital**.
- Nas próximas interações, a aplicação apresenta esse crachá para acessar os recursos permitidos.

Exemplo: entrada em um hospital

Na recepção, somos identificados conforme a finalidade da entrada: **paciente, acompanhante ou visitante**. Recebemos uma identificação que indica nosso vínculo com o hospital e ajuda a controlar o acesso às áreas permitidas para cada situação.

UNIVERSIDADE PAULISTA

O que é Hash?

Um resumo digital

Hash é o resultado de uma função que transforma um conteúdo em um **resumo digital de tamanho fixo**. O mesmo conteúdo gera o mesmo resumo, e pequenas alterações tendem a mudar completamente o resultado.

Não existe uma operação de "desfazer" o hash para recuperar o conteúdo original.

Exemplo: verificação de senhas

1. **No cadastro:** O sistema calcula e armazena um hash da senha, em vez de guardar a senha original.
2. **No login:** Usa a senha digitada para verificar se ela corresponde ao hash armazenado.
3. **Se corresponder:** A senha está correta, sem precisar recuperar a senha original.

Para senhas, são usados métodos próprios de hashing com um **salt**: um valor aleatório acrescentado ao processamento para dificultar tentativas de descoberta.

UNIVERSIDADE PAULISTA

Exemplo de Hash

Uma pequena mudança na entrada

Vamos calcular o hash de dois textos que diferem apenas na primeira letra: `ola` e `Ola`.

Texto: `ola`

```
55a9f4f8994b1bbf2058ea38c8efb6c459000814d5f39c087002571639e6230e
```

Texto: `Ola`

```
f21d8a4f1f93e8abd047bc92fc4f36be7a1a17da212c3833480466db2f192e6e
```

- Os dois resumos têm o **mesmo tamanho**.
- Mudar uma letra gerou resultados completamente diferentes.
- Repetir o cálculo para `ola` gera o primeiro resumo novamente.

Este exemplo mostra o hash de textos. Para armazenar senhas, usamos métodos próprios de hashing com salt.

UNIVERSIDADE PAULISTA

JWT: Visão Técnica

Um formato para transportar informações

JWT é um padrão (RFC 7519) que representa um conjunto de **claims**: informações expressas como pares de nome e valor em JSON.

- Em APIs, pode ser usado como **token de acesso** após a autenticação.
- A API valida o token antes de usar suas informações para aplicar as regras de autorização.
- Aqui usaremos um **JWT assinado (JWS)**: Seu conteúdo pode ser lido, mas alterações invalidam a assinatura.

UNIP
UNIVERSIDADE PAULISTA

Anatomia de um JWT Assinado

Header . Payload . Signature

Três partes em Base64url, separadas por dois pontos (.). As quebras de linha abaixo servem apenas para facilitar a leitura.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiI0MiIsInJvbGUiOiJ2aXNpdGFudGUiLCJpYXQiOjE3OTMyMjQ4MDAsImV4cCI6MTc5MzIyODQwMH0.
JuID693FlCiOeGbCImBUFAK9MfqNz0IgBPqI2ozieUQ
```

- **Header:** Descreve o tipo e o algoritmo.
- **Payload:** Contém as informações (claims).
- **Signature:** Permite verificar integridade e origem.

Exemplo didático gerado com HS256. Base64url é codificação, não criptografia.

UNIP
UNIVERSIDADE PAULISTA

Header: Cabeçalho

Descreve como o token foi assinado

A primeira parte contém metadados sobre o token.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- `alg`: Algoritmo utilizado. **HS256** corresponde a HMAC com SHA-256.
- `typ`: Indica o tipo do token, aqui **JWT**.
- Esse JSON é codificado em **Base64url** para formar a primeira parte.

O algoritmo deve ser permitido pela API

A API deve aceitar apenas algoritmos configurados como confiáveis; não basta seguir o valor recebido em `alg`.

Payload: Informações do Token

```
{
  "sub": "42",
  "role": "visitante",
  "iat": 1793224800,
  "exp": 1793228400
}
```

- `sub`: Identifica o sujeito do token; aqui, o usuário 42.
- `role`: Perfil definido pela aplicação, ligado às regras de acesso.
- `iat`: Emissão; `exp`: Expiração. Valores em segundos desde 01/01/1970, em UTC.

Exemplo: emissão em 28/10/2026 às 19h; expiração às 20h (Brasília).

O payload pode ser lido

Não inclua senhas ou segredos em um JWT apenas assinado. Use as claims somente após validar o token.

Signature: Assinatura

Protege o cabeçalho e o payload contra alterações

No HS256, o emissor calcula um HMAC usando uma chave secreta compartilhada com a API que valida o token.

```
entrada = base64url(header) + "." + base64url(payload)
assinatura = base64url(HMAC_SHA256(chave, entrada))
```

- A API recalcula o HMAC e compara com a assinatura recebida.
- Alterar `role` de `visitante` para `admin` invalida a assinatura original.
- A chave **não faz parte do token** e deve permanecer no servidor.

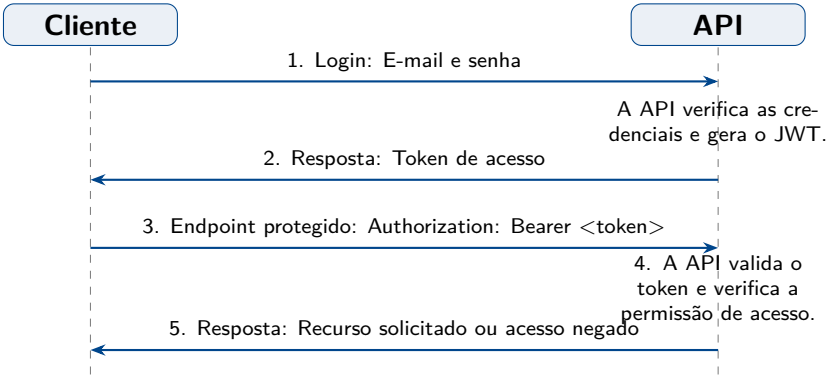
Assinatura válida é uma etapa da validação

Também verifique expiração, emissor e destinatário esperados, conforme as regras da aplicação.

UNIVERSIDADE PAULISTA

Fluxo de uso do JWT em uma API

O cliente obtém o token no login e o apresenta ao acessar um endpoint protegido.



O token é enviado novamente em cada requisição protegida. Token ausente ou inválido resulta em **401**; usuário autenticado sem permissão recebe **403**.

Implementação de JWT

Nesta prática, será implementada a autenticação com JWT no projeto base **ExemploJwt**.

A implementação terá início com a validação do login. Em seguida, será acrescentada a geração do token, que será apresentado nas requisições para identificar o usuário e controlar o acesso aos endpoints.

UNIP
UNIVERSIDADE PAULISTA

Modelos do Projeto Base

Na pasta `Models` do projeto **ExemploJwt**, já estão criados os modelos abaixo.

Usuario

```
public class Usuario {
    public int Id { get; set; }
    public string Email { get; set; } = "";
    public string Senha { get; set; } = "";
    public string Role { get; set; } = "Usuario";
}
```

Produto

```
public class Produto {
    public int Id { get; set; }
    public string Nome { get; set; } = "";
}
```

UsuarioDto

```
public class UsuarioDto {
    public string Email { get; set; } = "";
    public string Senha { get; set; } = "";
}
```

Finalidade do AuthController

O `AuthController` será responsável por receber as requisições de login e verificar as credenciais do usuário.

- Receberá o e-mail e a senha enviados à API.
- Consultará os usuários mantidos em memória no próprio controller.
- Permitirá a emissão do token quando as credenciais forem válidas.
- Retornará uma resposta de acesso não autorizado quando o login falhar.

Em seguida, será implementado o método `Login` nesse controller.

UNIP
UNIVERSIDADE PAULISTA

Validação do Login

Deve-se inserir o método abaixo em `AuthController`.

```
[AllowAnonymous]
[HttpPost("login")]
public IActionResult Login(UsuarioDto dto)
{
    var usuario = _usuarios.FirstOrDefault(u =>
        u.Email.Equals(dto.Email, StringComparison.OrdinalIgnoreCase)
        && u.Senha == dto.Senha);

    if (usuario is null)
        return Unauthorized();

    return Ok(new { mensagem = "Login realizado com sucesso" });
}
```

Credenciais incorretas retornam 401. Neste passo, o login apenas verifica as credenciais; o token será acrescentado depois.

Teste do Login

Deve-se executar o projeto ExemploJwt e enviar a requisição abaixo ao endereço HTTPS da API.

```
POST /auth/login
Content-Type: application/json

{
  "email": "ana@aula.local",
  "senha": "Aula@123"
}
```

- Com as credenciais corretas, deve-se obter 200 e a mensagem de sucesso.
- Ao alterar a senha, deve-se obter 401.

Configuração do JWT

Deve-se acrescentar a seção abaixo ao `appsettings.json`:

```
"Jwt": {
  "Key": "ExemploJwt-chave-didatica-32-bytes-minimo!",
  "Issuer": "ExemploJwt",
  "Audience": "ExemploJwt.Cliente"
}
```

`Key`: chave de assinatura; `Issuer`: emissor; `Audience`: destinatário esperado.

Existem formas mais seguras de armazenar a chave, como User Secrets e gerenciadores de segredos.

UNIP
UNIVERSIDADE PAULISTA

Pacotes para a Implementação

Devem-se instalar os seguintes pacotes pelo gerenciador NuGet do projeto **ExemploJwt**:

- `Microsoft.AspNetCore.Authentication.JwtBearer`: Integra a autenticação Bearer ao ASP.NET Core e valida os tokens recebidos pela API.
- `System.IdentityModel.Tokens.Jwt`: Fornece `JwtSecurityToken` e `JwtSecurityTokenHandler`, utilizados na geração do token.

A dependência `Microsoft.IdentityModel.Tokens` será restaurada automaticamente e fornece os tipos de chave, assinatura e validação.

Deve-se selecionar uma versão do `JwtBearer` compatível com o .NET do projeto e versões compatíveis entre os pacotes IdentityModel.

Criação do Serviço de Token

Deve-se criar `TokenService.cs` com a estrutura abaixo.

```
1  using System.IdentityModel.Tokens.Jwt;
2  using System.Security.Claims;
3  using System.Text;
4  using Microsoft.IdentityModel.Tokens;
5
6  public class TokenService
7  {
8      private readonly IConfiguration _config;
9
10     public TokenService(IConfiguration config)
11     {
12         _config = config;
13     }
14 }
```

`IConfiguration` é a interface usada para consultar as configurações da aplicação, como as do `appsettings.json`. O ASP.NET Core a fornece pelo construtor, por injeção de dependência. O campo `_config` guarda esse acesso; `_config["Jwt:Key"]` lê `Key` dentro da seção `Jwt`.

Geração do JWT

Deve-se inserir o método abaixo em `TokenService`.

```
public string Gerar(Usuario usuario)
{
    var chave = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(
        _config["Jwt:Key"]!));
    var claims = new[]
    {
        new Claim(JwtRegisteredClaimNames.Sub, usuario.Email),
        new Claim("role", usuario.Role)
    };
    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        notBefore: DateTime.UtcNow,
        expires: DateTime.UtcNow.AddMinutes(30),
        signingCredentials: new SigningCredentials(
            chave, SecurityAlgorithms.HmacSha256));

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

Geração do JWT: Método e Chave

Deve-se iniciar o método que receberá o usuário autenticado e gerará seu token.

```
1 public string Gerar(Usuario usuario)
2 {
3     var chave = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(
4         _config["Jwt:Key"]!));
```

- O comando lê a chave definida em `Jwt:Key` no `appsettings.json` e a prepara para uso na assinatura do token.
- O texto da chave é convertido em bytes e utilizado para criar um objeto `SymmetricSecurityKey`.
- Essa chave permitirá assinar o token. A API utilizará a mesma chave para verificar se ele foi alterado.

Neste ponto, apenas a chave foi preparada; o token será montado nos próximos passos.

Geração do JWT: Claims do Usuário

Devem-se definir as informações do usuário que serão incluídas no payload.

```
1    var claims = new[]
2    {
3        new Claim(JwtRegisteredClaimNames.Sub, usuario.Email),
4        new Claim("role", usuario.Role)
5    };
```

- `Claim`: Representa uma informação no formato nome e valor, incluída no payload do token.
- `JwtRegisteredClaimNames.Sub`: Corresponde à claim padronizada `sub` (subject), que identifica o usuário a quem o token se refere; neste exemplo, pelo e-mail.
- `role`: Claim definida pela aplicação para indicar o perfil, como `Usuario` ou `Admin`. Será usada nas regras de autorização.
- `claims`: Reúne essas informações para incluí-las na criação do JWT.

Os valores das claims são obtidos do usuário cujo login foi validado no servidor.

Geração do JWT: Emissor e Destinatário

Deve-se montar o token com sua origem, seu destinatário e as informações do usuário.

```
1    var token = new JwtSecurityToken(
2        issuer: _config["Jwt:Issuer"],
3        audience: _config["Jwt:Audience"],
4        claims: claims,
```

- `JwtSecurityToken`: Classe que representa o JWT que está sendo montado.
- `issuer`: Identifica quem emite o token e gera a claim `iss`. Neste exemplo, o valor é `ExemploJwt`.
- `audience`: Identifica para quem o token foi emitido e gera a claim `aud`. O valor configurado é `ExemploJwt.Cliente`.
- `claims`: Acrescenta ao payload as informações do usuário preparadas no trecho anterior.

A API validará as claims `iss` e `aud` para aceitar apenas tokens com o emissor e o destinatário esperados.

Geração do JWT: Validade e Assinatura

Deve-se definir por quanto tempo o token será válido e como ele será assinado.

```
1        notBefore: DateTime.UtcNow,
2        expires: DateTime.UtcNow.AddMinutes(30),
3        signingCredentials: new SigningCredentials(
4            chave, SecurityAlgorithms.HmacSha256));
```

- `notBefore`: Define o início da validade e gera a claim `nbf`. O token não deve ser aceito antes desse instante.
- `expires`: Define a expiração e gera a claim `exp`. Neste exemplo, o prazo é de 30 minutos.
- `signingCredentials`: Informa as credenciais utilizadas para assinar o token.
- `SigningCredentials`: Combina a chave secreta com o algoritmo de assinatura.
- `SecurityAlgorithms.HmacSha256`: Seleciona HS256, indicado no campo `alg` do header.

Os instantes são definidos em UTC. A assinatura permite detectar alterações; ela não oculta o conteúdo do token.

Geração do JWT: Retorno do Token

Deve-se transformar o token em texto para que ele possa ser enviado ao cliente.

```
    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

- `JwtSecurityTokenHandler`: Componente responsável por escrever o objeto JWT como um token em texto.
- `WriteToken(token)`: Produz o formato compacto do JWT assinado, calculando a assinatura com as credenciais configuradas.
- O resultado segue a estrutura `header.payload.signature`: Cabeçalho, informações e assinatura, separados por pontos.

Esse texto será devolvido ao controller para compor o `access_token` da resposta do login. O cliente poderá apresentá-lo como `Authorization: Bearer <token>` nas próximas requisições.

Inclusão do Token no Login

Deve-se injetar `TokenService` no `AuthController`, acrescentando o campo e o construtor abaixo.

```
private readonly TokenService _tokens;

public AuthController(TokenService tokens)
{
    _tokens = tokens;
}
```

Deve-se substituir o retorno de sucesso do método `Login` por:

```
return Ok(new { access_token = _tokens.Gerar(usuario),
    token_type = "Bearer", expires_in = 1800 });
```

A role incluída no token vem do usuário armazenado no servidor.

Registro do TokenService

Deve-se registrar o `TokenService` no `Program.cs`, antes de `builder.Build()`.

```
builder.Services.AddScoped<TokenService>();
```

- `AddScoped<TokenService>()`: Registra o serviço para que o ASP.NET Core possa fornecê-lo ao construtor do `AuthController`.
- O serviço é criado uma vez por requisição e compartilhado durante o processamento dessa requisição.
- Ao criar o serviço, o framework também fornece a dependência `IConfiguration` utilizada por ele.

Teste da Geração do Token

Deve-se executar o projeto e enviar o login ao endereço HTTPS da API.

```
POST /auth/login
Content-Type: application/json

{ "email": "ana@aula.local", "senha": "Aula@123" }
```

Deve-se verificar se a resposta retorna status **200 OK** e os campos abaixo:

```
{
  "access_token": "eyJ...eyJ...assinatura",
  "token_type": "Bearer",
  "expires_in": 1800
}
```

- `access_token`: Deve conter o JWT gerado, com três partes separadas por dois pontos. O valor acima está abreviado para ilustração.
- `expires_in`: Indica a validade de 1800 segundos, ou 30 minutos.
- Deve-se copiar o token para os próximos testes de acesso à API.

UNIP UNIVERSIDADE PAULISTA

Autenticação com Bearer

Será configurado o componente do ASP.NET Core responsável por receber o JWT e verificar se ele pode identificar o usuário da requisição.

- O cliente enviará o token no cabeçalho `Authorization: Bearer <token>`.
- A configuração definirá qual chave, emissor, destinatário e algoritmo a API considera confiáveis, além do período de validade.
- Após a validação, as claims do token formarão a identidade disponível em `HttpContext.User`, incluindo o perfil do usuário.

Deve-se realizar essa configuração no `Program.cs`, com `AddAuthentication` e `AddJwtBearer`. O fluxo será ativado depois com `UseAuthentication()`.

Registro dos Serviços de JWT

Devem-se acrescentar os namespaces e registros no `Program.cs`, antes de `builder.Build()`.

```
1       using System.Text;
2       using Microsoft.AspNetCore.Authentication.JwtBearer;
3       using Microsoft.IdentityModel.Tokens;
4
5
6       builder.Services.AddAuthorization();
7
8       var chave = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(
9       builder.Configuration["Jwt:Key"]
10      ?? throw new InvalidOperationException("Chave ausente")));
```

Configuração da Autenticação

Deve-se registrar o Bearer no `Program.cs`, após a criação de `chave` e antes de `builder.Build()`.

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.MapInboundClaims = false;
        options.TokenValidationParameters = new()
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = chave,
            ValidateIssuer = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidateAudience = true,
            ValidAudience = builder.Configuration["Jwt:Audience"],
            ValidateLifetime = true,
            ClockSkew = TimeSpan.Zero,
            RoleClaimType = "role",
            ValidAlgorithms = new[] { SecurityAlgorithms.HmacSha256 }
        };
    });
```

Autenticação: Registro do Bearer

Deve-se registrar o mecanismo que identificará o usuário a partir do token.

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
```

- `AddAuthentication`: Prepara a aplicação para identificar quem está fazendo a requisição e escolhe Bearer como esquema padrão.
- `AddJwtBearer`: Adiciona o componente que recebe o JWT pelo cabeçalho `Authorization: Bearer <token>` e verifica sua validade.
- As opções configuradas a seguir determinam quais tokens a API aceitará.

Com essa configuração, a API poderá reconhecer o usuário a partir do token, sem solicitar e-mail e senha em cada chamada. O processamento será ativado com `UseAuthentication()`.

UNIVERSIDADE PAULISTA

Autenticação: Claims e Parâmetros

Deve-se definir como as claims serão tratadas e iniciar as regras de validação.

```
1        options.MapInboundClaims = false;
2        options.TokenValidationParameters = new()
3        {
```

- `MapInboundClaims = false`: Preserva os nomes das informações recebidas no token, como `sub` e `role`, para que a aplicação possa utilizá-los com esses mesmos nomes.
- `TokenValidationParameters`: Centraliza as condições que o token precisa atender para ser aceito: Assinatura, emissor, destinatário, prazo de validade e algoritmo.

Essas regras serão aplicadas pelo componente Bearer antes de utilizar as claims para identificar o usuário. Se alguma verificação falhar, o token não estabelecerá uma autenticação válida.

UNIP
UNIVERSIDADE PAULISTA

Autenticação: Chave e Emissor

Devem-se configurar a validação da assinatura e a identificação do emissor.

```
1            ValidateIssuerSigningKey = true,
2            IssuerSigningKey = chave,
3            ValidateIssuer = true,
4            ValidIssuer = builder.Configuration["Jwt:Issuer"],
```

- `ValidateIssuerSigningKey`: Habilita a validação da chave utilizada na assinatura.
- `IssuerSigningKey`: Define a chave confiável que permitirá conferir a assinatura e detectar alterações no token.
- `ValidateIssuer`: Faz a API conferir quem declarou ter emitido o token.
- `ValidIssuer`: Define o emissor esperado, lido de `Jwt:Issuer`; a claim `iss` precisa corresponder a esse valor.

A assinatura e o emissor são verificações complementares. Um token com conteúdo alterado ou emissor diferente do esperado será rejeitado.

Autenticação: Destinatário e Validade

Devem-se verificar o destinatário esperado e o período de validade do token.

```
1            ValidateAudience = true,
2            ValidAudience = builder.Configuration["Jwt:Audience"],
3            ValidateLifetime = true,
4            ClockSkew = TimeSpan.Zero,
```

- `ValidateAudience`: Faz a API verificar se é um destinatário previsto para aquele token.
- `ValidAudience`: Define o destinatário aceito, lido de `Jwt:Audience`, para comparação com a claim aud.
- `ValidateLifetime`: Impede o uso de um token expirado ou cujo período de validade ainda não começou.
- `ClockSkew = TimeSpan.Zero`: Remove a tolerância de horário; o token não recebe tempo extra depois da expiração.

Mesmo com assinatura válida, um token destinado a outro serviço ou fora do período permitido não será aceito.

Autenticação: Perfil e Algoritmo

Deve-se indicar a claim de perfil e restringir o algoritmo aceito pela API.

```
            RoleClaimType = "role",
            ValidAlgorithms = new[] { SecurityAlgorithms.HmacSha256 }
        };
    });
```

- `RoleClaimType = "role"`: Permite ao framework reconhecer o perfil do usuário a partir da claim `role`.
- Esse perfil será utilizado pelas regras dos endpoints. Por exemplo: `[Authorize(Roles = "Admin")]` exigirá o perfil Admin.
- `ValidAlgorithms`: Restringe quais algoritmos de assinatura a API aceita. Neste exemplo, apenas HS256 é permitido.

Reconhecer a role prepara a autorização; a restrição de acesso será aplicada no endpoint. A limitação do algoritmo evita aceitar uma forma de assinatura diferente da prevista pela aplicação.

Ativação da Autenticação no Fluxo

Deve-se finalizar o `Program.cs` nesta ordem.

```
var app = builder.Build();

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
app.Run();
```

- `UseAuthentication()`: Valida o token e identifica o usuário.
- `UseAuthorization()`: Verifica as regras de acesso do endpoint.
- `RoleClaimType = "role"`: Faz o framework reconhecer a claim usada para os perfis.

Endpoints do Projeto Base

Deve-se observar o `ProdutosController`, já presente no projeto ExemploJwt.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("produtos")]
public class ProdutosController : ControllerBase
{
    private static readonly List<Produto> _produtos = new()
    {
        new Produto { Id = 1, Nome = "Teclado" }
    };

    // Os tres endpoints existentes serao ajustados a seguir.
}
```

UNIP
UNIVERSIDADE PAULISTA

Aplicação das Regras de Acesso

Devem-se aplicar as regras aos três endpoints do controller.

```
[AllowAnonymous]
[HttpGet("publico")]
public IActionResult Publico() => Ok(_produtos);

[Authorize]
[HttpGet("autenticado")]
public IActionResult Autenticado() =>
    Ok(new { mensagem = "Acesso autenticado", produtos = _produtos });

[Authorize(Roles = "Admin")]
[HttpGet("admin")]
public IActionResult Admin() =>
    Ok(new { mensagem = "Acesso administrativo", produtos = _produtos });
```

`[Authorize]` exige autenticação; `[Authorize(Roles = "Admin")]` exige também a role indicada.
Os nomes dos perfis devem coincidir exatamente.

UNIP
UNIVERSIDADE PAULISTA

Configuração do Swagger para Testes com JWT

```
builder.Services.AddSwaggerGen(options =>
{
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        Description = "Informe o token retornado pelo login."
    });
    options.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference
                {
                    Type = ReferenceType.SecurityScheme, Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
    });
});
```

UNIP
UNIVERSIDADE PAULISTA

Obtenção e Envio do Token

Deve-se enviar a requisição abaixo pelo cliente HTTP, utilizando o endereço HTTPS da API em execução.

```
POST /auth/login
Content-Type: application/json

{
  "email": "ana@aula.local",
  "senha": "Aula@123"
}
```

Em seguida, deve-se copiar o valor de `access_token` e enviá-lo na chamada ao endpoint protegido:

```
GET /produtos/autenticado
Authorization: Bearer <access_token>
```

O marcador `<access_token>` deve ser substituído pelo token recebido, sem os sinais de menor e maior.

UNIP
UNIVERSIDADE PAULISTA

Verificação da Autorização

Devem-se chamar os três endpoints nas situações abaixo.

Requisição	**/publico**	**/autenticado**	**/admin**
Sem token	200	401	401
Token de Usuario	200	200	403
Token de Admin	200	200	200

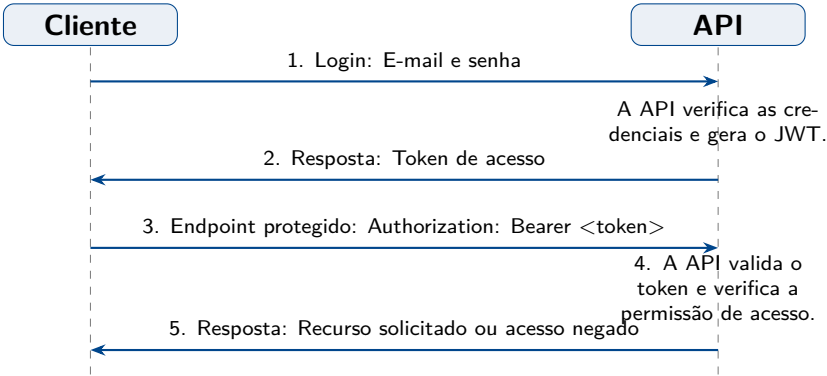
- Para testar Admin, deve-se repetir o login com `admin@aula.local` e senha `Admin@123`.
- **401**: Falta uma autenticação válida.
- **403**: O usuário está autenticado, mas não possui a role exigida.
- Um token alterado ou expirado deve retornar 401 nos endpoints protegidos.

A role Admin não herda automaticamente outras roles; o acesso ao endpoint autenticado ocorre porque ele aceita qualquer usuário autenticado.

UNIP
UNIVERSIDADE PAULISTA

Revisão do Fluxo com JWT

O cliente obtém o token no login e o apresenta ao acessar um endpoint protegido.



O token é enviado novamente em cada requisição protegida. Token ausente ou inválido resulta em **401**; usuário autenticado sem permissão recebe **403**.

Resumo Geral da Aula

- JWT representa o acesso do usuário após o login.
- O token assinado contém header, payload e signature.
- As claims carregam identificação, perfil e validade.
- A assinatura permite detectar alterações, mas não oculta o conteúdo.
- O login valida as credenciais e o TokenService gera o token.
- O cliente envia o JWT em `Authorization: Bearer <token>`.
- A API valida assinatura, emissor, destinatário e prazo de validade.
- A autenticação identifica o usuário; a autorização controla o acesso por roles.
- Os testes no Swagger verificam os acessos: 200 permitido, 401 sem autenticação válida e 403 sem permissão.

UNIP
UNIVERSIDADE PAULISTA

Obrigado!

Dúvidas?

UNIP
UNIVERSIDADE PAULISTA